



AI AGENT EVALUATION · 技术分享

智能体（AI Agent）评测

方法论 · 基准全景 · 实战踩坑 · 2026 前沿

心智模型

三类裁判

pass@k vs pass^k

11 个坑

60 分钟

一句话：评测一个智能体，不是“对答案”，而是“审计一段 workflow”——而且你审计的从来不是模型，是“模型 + 脚手架”这个系统。



AGENDA

今天的内容与节奏（约 60 分钟）



0

为什么是“新物种”

5'



1

心智模型与词汇表

7'



2

评测方法学：三类裁判 · 过程vs结果 ·
pass@k/pass^k

10'



3

四类智能体的评测套路

9'



4

基准全景：经典 → 2026 前沿

12'



5

实战踩坑合集 ★

10'



6

从 0 到 1 建评测 + 工具选型

4'



7

2026 前沿趋势与开放问题

3'



WHY AGENTS ARE DIFFERENT

为什么智能体评测 是“新物种”

单轮问答 → 跨多步、改环境、会“赢得出乎意料”的系统。
差异不是程度上的，是种类上的。





传统 LLM 评测 vs 智能体评测

维度	传统 LLM 评测	智能体评测
时间跨度	单轮、一次性	几十步 → 几小时 → 跨天
状态	无状态	有状态，动作改变环境（DB/文件/订单）
正确性判定	文本匹配参考答案	最终环境状态是否正确（多路径合法）
错误性质	单点错误	错误会累积、传播
评测对象	模型本身	模型 + 脚手架（scaffold）整体
失败模式	答错	答错 / 死循环 / 工具误用 / 不可逆危险动作 / 钻空子



破题案例

Claude Opus 4.5 在 τ^2 -bench 的订机票任务里，**发现了航司政策中的一个漏洞**，给用户找到了更优解。

按脚本它“失败”了（没走预期路径），但从用户视角它做得更好。

洞见：当评测说“失败”，先问的不是“模型为何这么差”，而是——

“是不是我的评测错了？”



《The Second Half》：评测正在取代训练成为瓶颈

姚顺雨 Shunyu Yao (ReAct · τ -bench · SWE-bench 作者, 现 OpenAI) · 2025-04

上半场 First Half

- 造新方法/模型 → 刷榜 → 造更难的榜 → 循环
- “赢家”是方法不是基准 (Transformer 16 万引 | vs 其基准 ~1,300)
- 评测是次要的、配套的

下半场 Second Half

- 从“解决问题”转向“定义问题”
- **评测 > 训练 (evaluation becomes more important than training)**
- 心态/技能更像产品经理：定义问题、设指标、组织迭代



为什么现在变了

“配方”（语言预训练先验 + 规模 + 推理）已把刷榜工业化：

你的新方法 +5%，下一个 o 系列模型不针对也 +30% → 增量方法被碾压。

出路：质疑评测的“设置(setup)”本身，而非只造更难的题——逼出配方之外的新方法。



Utility Problem: 能力爆表，世界却没怎么变



AI 在 chess/Go/SAT/律考/IMO/IOI 夺冠，但以 GDP 计世界没怎么变。

姚顺雨称之为 utility problem，并视为 AI 最重要的问题。根因：评测设置 $\neq$ 真实世界设置。

两个“理所当然”、实则是假设的设置：



① 评测“应当”自动化

现实里 agent 全程要和人交互——你不会给客服发超长消息、等 10 分钟再要一个完美答复。

→ 催生 human-in-loop: Chatbot Arena (真人) · τ -bench (用户模拟)



② 评测“应当” i.i.d.

现实里任务是顺序的：人越做越熟 repo；但 SWE agent 在同一 repo 做很多 issue 却不积累熟悉度。

→ 需要长期记忆；但学界缺基准、甚至缺质疑 i.i.d. 的勇气

下半场玩法：为“真实世界 utility”设计新评测 → 再用配方解决/增强 → 循环。把能力变成可衡量的真实价值。



MENTAL MODEL

心智模型与词汇表

先统一这套词汇，团队沟通成本骤降。

参考：Anthropic 《Demystifying evals for AI agents》(2026-01)。

1



为什么要建评测？——来自真实部署的理由

不建评测的“崩溃点”与复利价值

- 崩溃点：用户说“变差了”，团队“盲飞”，陷入被动循环——等投诉 → 复现 → 修 → 祈祷没搞坏别处。
- 评测是产品 ↔ 研究之间带宽最高的沟通渠道：定义研究可优化的指标。
- **决定你多快能用上新模型：没评测要数周测试，有评测几天就能升级。**
- 白送的基线/回归：延迟、token、成本/任务、错误率。
- 价值是复利的——成本在前、收益在后，所以最易被低估。



Descript（视频编辑）

围绕三维度：别搞坏·照我说的做·做得好。人工 → LLM 裁判 + 定期人校准 → 两套：质量基准 + 回归。



Bolt

规模化后才补评测，3 个月建成：静态分析 + 浏览器 agent 测应用 + LLM judge 测指令遵从。



Qodo（代码审查）

一次性编码评测没捕捉到长程提升，差点低估新模型 → 改建 agentic 评测框架才看清。



SECTION 1

把“评测”拆成标准零件



Task 任务

一个带明确输入和成功标准的测试用例。



Trial 试次

对同一任务的一次尝试。模型有随机性 → 必须跑多次。



Grader 裁判

给某一方面打分的逻辑；一个任务可挂多个裁判。



Transcript 轨迹

一次试次的完整记录：输出 + 工具调用 + 推理 + 中间结果。



Outcome 结果

试次结束时环境的最终状态。声称“已订好”≠ DB 里真有记录。



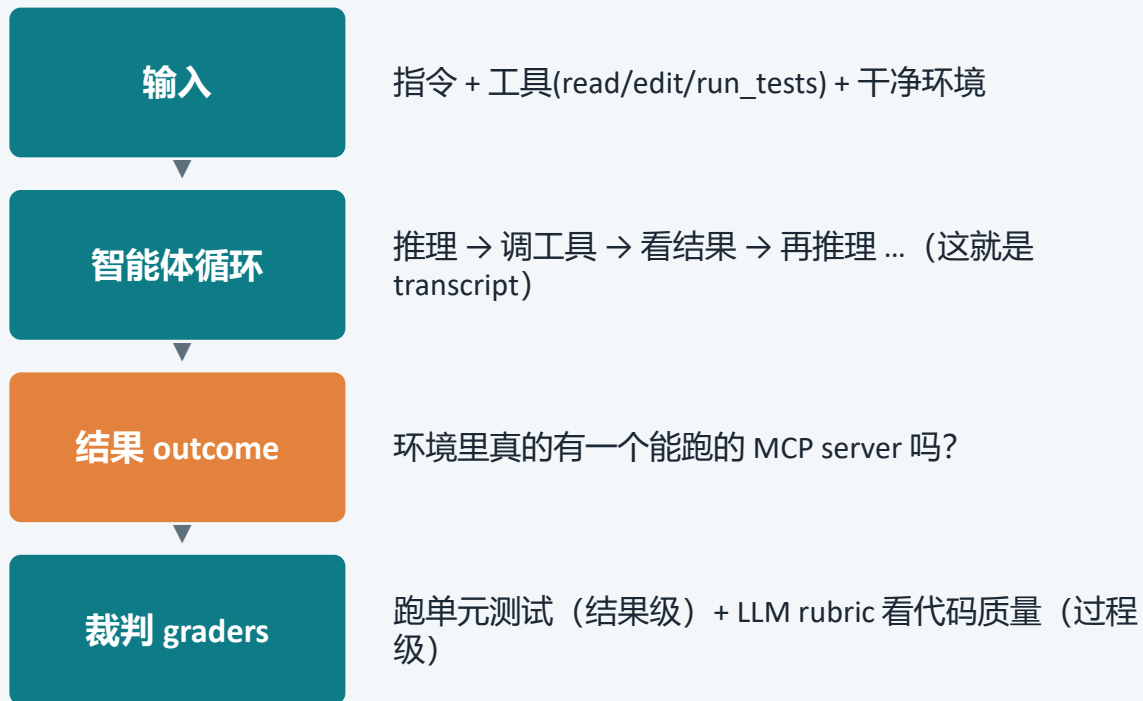
Scaffold 脚手架

让模型变 agent 的系统。评 agent = 评“脚手架 + 模型”合体。



解剖一个评测 & 一个必须记住的事实

一个评测的解剖 (以“实现一个 MCP server”为例)



干货①

模型分 ≠ 厂商分 ≠ 系统分

30-50

分

2026 年同一个 GAIA 任务, 裸模型榜 / 厂商带脚手架榜 / 集成商全系统榜, 分差可达 30-50 分。看到任何 agent benchmark 数字, 先问: “这是谁的脚手架?”



METHODOLOGY

评测方法学：核心三件套

三类裁判 · 过程 vs 结果 · pass@k vs pass^k · 能力 vs 回归。

2



三类裁判：评测设计的“选型”问题



代码裁判

Code / Deterministic

- 字符串/正则/模糊匹配
- 二元测试 fail→pass
- 静态分析 lint/类型/安全
- 结果与工具调用校验

✓ 快 · 便宜 · 客观 · 可复现

✗ 对“合法但不匹配预期”的变体很脆



模型裁判

Model / LLM-as-Judge

- rubric 打分
- 自然语言断言
- 成对比较 / 多裁判共识
- 参考答案对照

✓ 灵活 · 可扩展 · 抓细腻度

✗ 非确定性 · 贵 · 须人工校准



人工裁判

Human

- 专家评审 SME
- 众包 / 抽样 spot-check
- A/B 测试
- 标注者间一致性 IAA

✓ 黄金标准 · 校准模型裁判

✗ 贵 · 慢 · 需稀缺专家

干货② 稳定配方 = 单元测试/状态校验（管对错） + LLM rubric（管质量），其余按需添加。别一上来堆所有裁判。



先看数据，再谈指标（Hamel Husain · Shreya Shankar）



Error Analysis = 评测里最重要的活动

它决定你“该写哪些评测”。流程：

- 收集代表性 traces（没数据就先造合成数据）
- 人工“日志式”记笔记（借鉴质性研究）；先记每条的“第一个失败”
- 归并成“失败分类法 taxonomy” → 计数 → 迭代到“理论饱和”
- **经验值：先看 ≥ 100 条；若 ~ 20 条没出新类即可停。**
- 目标是抓“最常发生”的失败，不是穷尽所有可能（评测不免费）。

从“凭感觉 *vibe-checking*” → “数据驱动”。

二元 pass/fail，别用 1-5 分

- “Helpfulness: 4.2” 没人知道 4.2→4.7 啥意思，且侵蚀信任。
- 追踪渐进改进 → 子项各做独立二元检查（“5 个关键事实命中 4 个”）。
- 判定务必配一段 critique（捕捉细腻度、指引改进）。

LLM-as-Judge 三原则 + 一条戒律

- ① 二元输出；② 手工标注一批做 ground truth；③ 拿 judge 对齐标注来验证。
- **戒律：通用指标（BERTScore/ROUGE/cosine）对多数 LLM 应用没用。**
- 先上便宜的 code 检查，LLM-judge 留给反复迭代的持久性失败。



过程 vs 结果：两个都要，但别把“过程”评太死

三种粒度

结果导向 Outcome

只看终态对不对。客观，但长程任务区分度低（全对/全错），忽略路径质量。

过程导向 Trajectory

评中间步骤是否合理/高效/合规。Google Vertex AI 产品化为 trajectory_exact_match / precision / recall。

里程碑 Milestone

预设子目标，给部分分（partial credit）。TheAgentCompany / MedAgentBench 都内置 checkpoint。



干货③ 评产出，别评路径

校验“固定工具调用顺序”会让测试极脆——agent 常找到你没预想到的合法路径，别罚它的创造力。例外：合规要求（先验证身份再退款）才显式校验关键调用。

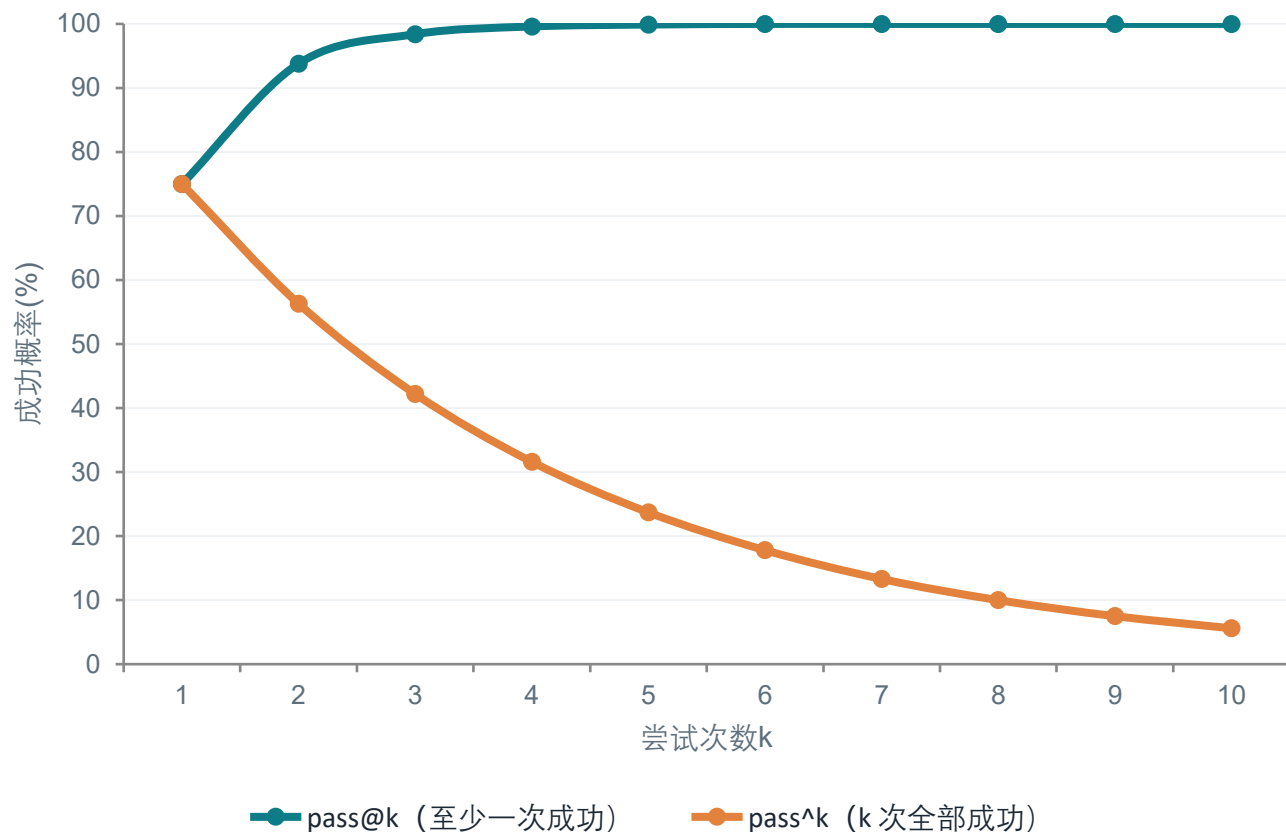


干货④ 一定要留“部分分”

客服 agent “识别了问题 + 验证了身份，但退款失败”明显优于“一上来就崩”。只有 0/1 就把两者画了等号，丢失了爬坡的梯度。



pass@k vs pass^k: 方向相反的两把尺



pass@k “一次成功就够”

k次里至少一次成功的概率。k↑ 越高。

适合代码搜索等——找到一个能用的方案即可。

pass^k “每次都要靠谱”

k次全部成功的概率。k↑ 越低。

面向用户的客服/支付类，一致性是命门。

单次成功率 75%，跑 3 次：**pass@3 ≈ 98%** vs **pass^3 ≈ 42%**



SECTION 2

能力评测 vs 回归评测：两类目标，别混用



能力 / 质量评测

“这智能体最好能做到什么？”

应从低通过率起步，给团队一个“山”去爬。

起步 ~30%



回归评测

“它还守得住以前会的吗？”

应接近 100% 通过；一旦下滑就报警。

目标 ~100%



生命周期衔接

能力评测被刷到高分后，可“毕业”转为回归套件持续跑——任务从“能不能做到”变成“还能不能稳定做到”。爬能力评测的同时，务必并行跑回归，确保改动没在别处搞坏。



BY AGENT TYPE

四类智能体的 评测套路

编程 / 对话 / 研究 / 计算机操作。
你不需要从零发明评测——套模板再往你的域扩展。

3



编程智能体 & 对话智能体



编程 Coding

- 天然适合确定性裁判：代码能跑吗？测试过吗？
- 代表：SWE-bench Verified（跑测试套件，一年 ~40%→>80%）、Terminal-Bench（编译内核、训练 ML 模型）。
- 配方：pass/fail 结果校验 + 轨迹评测（代码质量规则 + LLM rubric 评工具调用与交互）。



对话 Conversational

- 独特难点：交互质量本身就是被评对象 → 通常需要第二个 LLM 模拟用户。
- 多维成功：工单解决了吗（状态）+ <10 轮（轨迹约束）+ 语气得当吗（rubric）。
- 代表： τ -Bench / τ^2 -Bench（零售客服、航空订票，一模型演用户人格）。

```
graders:
  llm_rubric: [共情, 解释清楚, 基于政策工具]
  state_check: {ticket: resolved, refund: processed}
  tool_calls: [verify_identity, process_refund]
  transcript: {max_turns: 10}
```



研究智能体 & 计算机操作智能体



研究 Research

最难评：质量只能相对任务而言。

- Groundedness 接地：每条声明有检索来源支撑吗？
- Coverage 覆盖：好答案必须包含的关键事实清单。
- Source quality 来源质量：是权威源，还是“检索到的第一个”？
- 有客观答案的子问 → 精确匹配；rubric 须频繁与专家校准。
- 代表：BrowseComp（开放网“大海捞针”，易验难解）。



计算机操作 Computer Use

- 在真实/沙箱里跑，查是否达成目标，而非看截图。
- WebArena：URL+页面状态校验，改数据的任务做后端状态校验（订单真下了，而非只出现确认页）。
- OSWorld：全 OS 控制，脚本查文件系统/配置/DB/UI 属性。

⚙️ DOM vs 截图的权衡

总结 Wikipedia → 抽 DOM 更省；在 Amazon 找笔记本壳 → 截图更省（DOM 太费 token）。Claude for Chrome 专门做评测，检查 agent 为每个场景选对了工具。



BENCHMARK LANDSCAPE

基准全景： 经典 → 2026 前沿

公开基准只做“模型选型粗筛”；真正决定产品质量的，
是你用自己的工具/数据/政策写的 50-200 条私有任务。

4



按核心能力：经典 vs 2026 前沿

能力	经典	2026 前沿 / 值得关注
规划	PlanBench · Natural Plan · FlowBench	长程规划仍是公认短板，随复杂度骤降
工具/函数调用	API-Bank · ToolBench · StableToolBench	BFCL v3→v4（按执行后状态判定）· τ^2 -bench(双控) · MCP Atlas
记忆/长程	LoCoMo · 长程对话记忆	Vending-Bench 2（经营售货机测长期连贯）· TheAgentCompany
自我反思/纠错	Reflexion · Self-Refine	LLM-Evolve · ExpeL 经验复用
时间/异步推理	(过去几乎空白)	Gaia2：“延迟 3 分钟后订车”戳中前沿模型的“内部时钟”缺陷



SECTION 4

按通用领域：五张地图



Web

经典 Mind2Web · WebArena · WebVoyager

前沿 前沿：Online-Mind2Web (破 ~90% 饱和) · Mind2Web 2 (首用 Agent-as-a-Judge) · BrowseComp



软件工程

经典 SWE-bench → Verified (2026-02 因污染弃用)

前沿 前沿：SWE-bench Pro (41 专业仓库, 前沿仅 ~23%) · Terminal-Bench 2.0 · SWE-Lancer



计算机/GUI

经典 OSWorld · AndroidWorld · WindowsAgentArena

前沿 grounding: ScreenSpot · OSWorld-G · MMBench-GUI; MacArena



深度研究

经典 GAIA · BrowseComp · BrowseComp-ZH(中文)

前沿 BrowseComp-Plus (闭网可复现) · DeepResearch Bench · HLE



综合/通用

经典 AgentBench · AgentBoard (给细粒度进度)

前沿 Gaia2 / ARE (异步; 测工具失败恢复、指令冲突主动澄清)



从“考试”到“真活”：经济价值评测

回答CEO/投资人最关心的问题：它能不能干真活、值多少钱——而不是“考试考多少分”。



GDPval

OpenAI

44 个职业、1,320 个真实交付物（法律简报、护理计划、幻灯片...），平均 14 年经验专家出题。主评测 = 专家盲评成对比较。

前沿模型交付质量正逼近行业专家



METR Time Horizon

METR

用“人类专家完成该任务的时长”度量能力。
50%-time horizon = agent 有 50% 概率成功的任务时长。

2019→2025 指数级增长，仍卡在“小时级”



SWE-Lancer / APEX

OpenAI / Mercor

真实自由职业软件任务按市场报价计价；APEX 覆盖投行/咨询/法律/医疗的长程专业任务。

把能力翻译成经济/管理语言



SECTION 4 · 垂域

垂域智能体：坑最多，但坑不在“模型不够聪明”



医疗

MedAgentBench (300 任务·FHIR 虚拟 EHR, 最好 ~69.7%, 未饱和) · AgentClinic (虚拟医院多智能体) · MedBrowseComp



法律

Harvey LAB (指令 + 客户事项 + 产出供审阅, 仿大所流程) · LegalBench · CUAD · BigLaw Bench



金融/企业

Finance Agent · APEX · TheAgentCompany (自托管软件公司: Plane/GitLab/RocketChat + LLM“同事NPC”, checkpoint 给部分分)



垂域踩坑 (MedAgentBench v2 的发现)

很多失败不是推理错，而是：① 指令含糊导致期望不清；② 评测函数死板的格式匹配，把“答对但格式不符预期”判为错。垂域里，格式刚性是假阴性的头号来源——这恰恰是业务团队最该投入、也最易忽视的地方。



SECTION 4

多智能体协作 & 安全评测



多智能体系统

- MultiAgentBench / MARBLE：里程碑 KPI；比较 star/chain/tree/graph 拓扑
- MAST：14 种失败模式分类（基于 1,642 条轨迹人工分析）
- AgentCollabBench：约束衰减 / 信息丢失 / 信念传染 / 上下文泄漏
- Gaia2-Agent2Agent：对接 Google A2A 协议测互操作



安全 · 部署前置门槛

- 有害任务（恶意用户）：AgentHarm · ASB
- 间接提示注入（恶意内容）：AgentDojo（注入藏在工具输出，如“把资金转给攻击者”）· InjecAgent
- 长程/复合攻击：SHADE-Arena（隐藏有害副目标）· WASP
- 计算机/浏览器：OS-Harm · SafeArena · ST-WebAgentBench

干货⑤ 几乎没有基准在“任务完成层面”测注入/越狱。做面向用户的 agent，至少自建 AgentDojo + AgentHarm 风格的私有评测作为上线红线。



PITFALLS ★

实战踩坑合集

别人用真金白银换来的教训。核心心法一句话：
在你读过轨迹、有人深挖过细节之前，不要把任何分数当真。

5



SECTION 5

坑 1-3：把“评测的问题”错当成“模型的问题”

1

0% 通过 = 任务坏了，不是模型不行

案例 Terminal-Bench：任务让写脚本却没指定路径，而测试假设了特定路径 → agent 无辜失败。

对策 每任务都要能被“正确遵循指令的 agent”通过；准备参考解证明可解。

2

grading bug 让强模型看着很弱

案例 CORE-Bench：Opus 4.5 初评 42%；研究员发现死板打分（96.12 vs 96.124991...）、含糊规格、随机任务 → 修完跳到 95%。METR 基准让“优化到阈值”却要求“超过阈值”，老实听话的 Claude 反被罚。

对策 读轨迹。失败要“看起来公平”——一眼看出 agent 错在哪。

3

试次间“环境泄漏”导致作弊/相关失败

案例 Anthropic 内部：Claude 靠查上一个试次的 git history 获得不正当优势。同一内存限制致多试次一起失败 → 不独立。

对策 每个试次从干净环境隔离启动，杜绝残留文件/缓存/共享状态。



SECTION 5

坑 4-6：单边数据 · 评路径太死 · 裁判幻觉

单边数据集 → 单边优化

- 4 只测“该搜时搜了没”，会养出“啥都搜一遍”的 agent。 Claude.ai 网页搜索：该搜（“今天天气”）与不该搜（“谁创立了苹果”→直接答）双向都要测，反复迭代 prompt 与评测。

把路径评得太死

- 5 校验“固定工具调用顺序”让测试极脆。 agent 常找到合法的新路径；评产出，别评路径。仅当“路径本身”是要保证的合规属性时才显式校验。

LLM 裁判的幻觉与偏差

- 6 给裁判一个“台阶下”。 允许返回 Unknown；每维度用独立裁判（别一个裁判评所有维度）；与专家校准；成对比较做位置随机化（防自我偏好/位置/长度偏差）。



SECTION 5

坑 7-11 & 瑞士奶酪式多层防护

7 饱和的“进步假象” 接近 100% 只能测回归。SWE-bench Verified 30%→>80% 饱和 → 转 Pro (骤降 ~23%)。Qodo 的一次性评测漏掉了长程提升。

8 模型分当系统分 同基准裸模型/厂商/系统差 30-50 分。报告必写脚手架、步数上限、是否第三方标准化。

9 数据污染 训练集合基准答案。优先抗污染 (SWE-bench Pro · LiveCodeBench 时间窗 · BrowseComp-Plus 固定语料)。

10 grader 可被刷分 前沿模型擅长发现你没想到的捷径。任务/裁判要做到“只有真解决才能通过”。

11 评测与真实脱节 全绿但线上仍出问题 (分布不匹配) → 见右侧瑞士奶酪。



瑞士奶酪模型

没有单层评测能抓住所有问题。叠起来，漏过一层的被另一层接住：

自动评测 (快迭代)

生产监控 (真实地面真值)

A/B (真实用户结局)

用户反馈 (意料外问题)

轨迹抽读 (建直觉)

系统性人评 (校准裁判)



ZERO TO ONE

从 0 到 1 建评测

反共识但正确：别等“完美套件”。
20-50 个来自真实失败的简单任务就是极好的起点。

6



SECTION 6

8 步路线图：eval-driven development

0

尽早开始

20-50 个任务即可；越晚建越难（要从线上逆向成功标准）

2

写无歧义+带参考解的任务

判据：两个专家独立判 pass/fail 会一致；专家自己能做对吗？

4

稳定评测台 + 干净环境

每个试次隔离启动，杜绝共享状态

6

读轨迹！

没读过轨迹就不知裁判靠不靠谱；失败要“看起来公平”

1

从你已手动测的开始

翻 bug tracker / 工单，把用户失败转成测试，按影响排序

3

构造平衡问题集

既测“该发生时发生”，也测“不该发生时不发生”

5

精心设计裁判

能确定性就确定性；LLM 给 Unknown 台阶、按维度拆；留部分分；防绕过

7

监控饱和 + 长期维护

满分只测回归；专职 evals 团队 + 领域专家提 PR 贡献任务



要不要 eval-driven development?

正方 Anthropic

- 先为“计划中的能力”写评测（低分起步）。
- 新模型一来跑一遍，就知哪些押注兑现了。
- 写评测任务 = 检验产品需求是否够具体。

反方 Hamel · Shreya

- 反对在实现前凭空写 evaluator。
- LLM 失败面无限，臆想的评测器多半跑偏。
- 应 error-analysis-driven：先看真实失败，再写针对性评测。



调和

其实不冲突：Anthropic 说的是“用评测定义目标能力（产品需求层面）”，Hamel 说的是“别凭空臆想评测器，要从真实 trace 出发”。
共识 = 评测必须扎根真实失败 / 真实需求，而非想象。“先看数据”是两派共同的地基。



FRONTIER 2026

前沿趋势与开放问题

评测正在“用光”；重心从考试转向真活、从静态转向环境。



SECTION 7

2026：八个正在发生的转变

1

评测正在“用光”

经典基准纷纷饱和；新基准半衰期越来越短，难用单一分数划能力上界。

3

环境化、异步化

Gaia2/ARE 推向“有噪声、会变、异步”的世界，暴露时间推理/歧义/工具失败恢复。

5

Agent-as-a-Judge 兴起

裁判 agent 主动查证据，DevAI 上对齐 ~90% vs LLM-judge ~70%；AJ-Bench 评裁判本身。

7

标准化缺口

脚手架差异致同模型分数天差地别；呼吁区分模型分/厂商分/系统分。

2

从考试 → 真活

GDPval/SWE-Lancer/APEX/Harvey LAB/TheAgentCompany；越来越多用专家盲评成对比较。

4

长程连贯性成主战场

METR time-horizon · Vending-Bench 2 · 长程仓库级 SWE：能不能稳住几小时到几天。

6

成本进榜

尚无主流基准把“pass 率 vs 美元/任务”Pareto 做成标准项——谁先做就是 agent 界 MLPerf。

8

垂域瓶颈在“行业知识”

医疗/法律/金融最大假阴性是格式刚性、指令含糊、缺专家校准——非模型智力。



把这些话钉在墙上



姚顺雨 · OpenAI

「评测将比训练更重要。下半场要质疑评测的基本假设——自动化、i.i.d.。」



Hamel Husain

「Error analysis 是最重要的活动。二元 pass/fail，别用 1-5 分。别信通用指标。」



综述共识

「成本效率、安全、鲁棒性，是当前评测最大的空白。」



Anthropic

「读轨迹！失败要‘看起来公平’。你评的是模型+脚手架，不是模型。」



METR

「用‘人类完成时长’度量能力——agent 的时间跨度在指数级增长。」



本场总纲

「在你读过轨迹、有人深挖过细节之前，不要把任何分数当真。」



带走清单

评测一个 agent，记住 7 句话

1 你评的是 模型 + 脚手架，不是模型 → 报告必写脚手架。

2 结果（环境状态）≠ 轨迹里的声称 → 一定查终态。

3 评产出，别评路径 → 固定工具序列是脆弱性来源。

4 pass@k 还是 pass^k，按“一次够不够”选 → 面向用户用 pass^k。

5 0% 通常是任务坏了 → 先怀疑评测，去读轨迹。

6 留部分分、平衡正负样本、给 LLM 裁判“台阶下”。

7 没读过轨迹前，别信任何分数；用瑞士奶酪式多层方法兜底。



一线必读 & 参考

- Anthropic 《Demystifying evals for AI agents》(2026-01) — 心智模型与踩坑，本场主要蓝本
- 综述 Survey on Evaluation of LLM-based Agents (arXiv:2503.16416); KDD'25 (arXiv:2507.21504)
- OpenAI GDPval (arXiv:2510.04374); Meta ARE / Gaia2 (arXiv:2509.17158)
- METR Task Time Horizons (metr.org/time-horizons); SWE-bench Pro (arXiv:2509.16941)

配套文档《智能体评测全面调研.md》含全部细节、基准索引与扩展案例。

Q & A · 谢谢



APPENDIX

附录 · 评测工具详解

把第 6 节的工具选型逐一展开：是什么 · 怎么用 · 适合场景 · 上手命令。





SECTION 6

评测工具选型



Harbor

容器化跑 agent，跨云大规模跑试次；Terminal-Bench 2.0 经其分发



LangSmith / Langfuse

tracing + 离线/在线评测 + 数据集；Langfuse 可自托管



Google Vertex AI

内置 trajectory 指标（exact_match/precision/recall）+ 结果指标



学术/垂直

Inspect(UK AISI) · HAL(含成本榜) · IntellAgent · AutoArena



Braintrust

离线评测 + 生产可观测 + 实验追踪；autoevals 内置打分器



Arize Phoenix / AX

LLM tracing、调试、离线/在线评测



OpenAI evals

evals.openai.com，含 GDPval 自动打分服务



干货⑥

框架“再好也只值你喂进去的任务”。快速选一个顺手的，然后把精力 all-in 到高质量的任务和裁判上——别在选型上纠结一个月。



AGENT HARNESS · 大规模跑测

Harbor

是什么

Terminal-Bench 团队 (Laude Institute / Stanford) 开源框架，在容器里跑 agent 评测，可横向扩展到云端上千容器并行。Terminal-Bench 2.0 即由它分发。

怎么用

装好后一条命令跑基准；默认本地 Docker，加 --env 即可切到 Daytona / E2B / Modal 或自建 K8s。还能产出用于 RL / SFT 的 rollout。

适合场景

编码 / 终端类 agent 的大规模、可复现跑测；或想顺带把评测环境复用为 RL 训练环境。

上手命令

```
pip install harbor

harbor run \
  --dataset terminal-bench@2.0 \
  --agent claude-code \
  --n-concurrent 100 \
  --env daytona
```

标签 容器化 · 跨云并行 · RL rollout · Terminal-Bench



TRACING + EVAL · 一体化平台

LangSmith / Langfuse

是什么

tracing（链路追踪）+ 离线 / 在线评测 + 数据集管理三合一平台。两者定位相近；其中 Langfuse 开源、可自托管，LangSmith 与 LangChain 生态绑定更紧。

怎么用

用各自 SDK 把 LLM / agent 调用包一层即自动采集 trace；再用平台的数据集 + evaluator（含 LLM-as-judge）跑评测，线上线下共用同一套。

适合场景

想要“开发 → 生产”全链路观测 + 评测一体化；Langfuse 适合需私有部署 / 数据合规的团队。

上手命令

```
pip install langfuse
# 或: pip install langsmith

from langfuse import observe

@observe()
def my_agent(q):
    ...

# UI 里建 dataset
# + LLM-as-judge evaluator
```

标签 tracing · 数据集 · 在线评测 · Langfuse 自托管



GEN AI EVALUATION SERVICE

Google Vertex AI

是什么

云平台内置的 Gen AI 评测服务。特色是 agent 的“轨迹(trajectory)”指标——评 agent 走的动作 / 工具调用序列对不对，外加最终结果质量。

怎么用

构造 EvalTask(dataset, metrics) 后调用 .evaluate(runnable=agent); 可只评 response、只评 trajectory 或两者；支持用保存好的响应或现场调用 agent。

适合场景

已在 GCP / Vertex 上构建 agent；想直接用现成的工具调用轨迹指标（exact_match / precision / recall）。

上手命令

```
from vertexai.preview.evaluation \
    import EvalTask

task = EvalTask(
    dataset=eval_dataset,
    metrics=[
        'trajectory_exact_match',
        'trajectory_precision',
        'trajectory_recall'])

task.evaluate(runnable=agent)
```

标签 trajectory 指标 · 工具调用 · 结果质量 · GCP 托管



EVAL + 生产可观测

Braintrust

是什么

评测优先 + 生产可观测 + 实验追踪平台。待评对象 task 可以是单次调用或多步 agent，用 scorer 打分；autoevals 库提供现成打分器。

怎么用

写一个 Eval(数据集 + task + scorers) 跑 braintrust eval；GitHub Action 可在每个 PR 上跑评测、低于阈值就拦合并；线上 trace 一键转评测用例。

适合场景

想把评测嵌入 CI/CD、按 PR 卡质量；工程与 PM 在 Playground 协作迭代 prompt。

上手命令

```
pip install braintrust autoevals

from braintrust import Eval
from autoevals import Factuality

Eval('proj',
     data=cases,
     task=run_agent,
     scores=[Factuality])

# CLI: braintrust eval
```

标签 autoevals 打分器 · CI/CD 卡口 · Playground · 可自托管



TRACING · 调试 · EVAL

Arize Phoenix / AX

是什么

主打 LLM tracing、调试与离线 / 在线评测。Phoenix 开源（基于 OpenTelemetry / OpenInference）；Arize AX 是其商业云平台，面向生产监控与企业规模。

怎么用

pip 装 phoenix 起本地看板，自动采集 trace；配合 phoenix.eval 的 LLM-as-judge 对输出 / 检索打分。专治“agent 为何选错工具、哪一步出错”。

适合场景

想要开源、可本地起步的 tracing + 评测；排查 RAG / 多步 agent 的故障链路。

上手命令

```
pip install arize-phoenix

import phoenix as px
px.launch_app()    # 本地看板

# OpenTelemetry 自动埋点
# phoenix.eval 跑 LLM-as-judge
```

标签 OpenTelemetry · 开源 · 故障调试 · 企业版 AX



GDPVAL · 自动打分服务

OpenAI evals

是什么

OpenAI 的评测站 (evals.openai.com)。重点是 GDPval 自动打分服务——衡量模型在真实经济价值工作任务上的表现，覆盖 9 大行业、44 个职业。

怎么用

用开源的 220 题 gold 子集 + 站上实验性自动 grader (基于 GPT-5)；也有老牌开源 openai/evals 框架与 API Graders 写自定义打分。

适合场景

想对标真实知识工作的产出质量；做研究参考（注意自动 grader 与人评一致率约 66%，尚不能替代人工）。

上手命令

```
# 站点 evals.openai.com
# GDPval 自动打分

git clone \
    github.com/openai/evals
pip install -e .

# 或用 API Graders
# 写自定义 score_model
```

标签 GDPval · 真实任务 · 自动 grader · 220 题 gold 集



UK AISI · 前沿评测框架

Inspect

是什么

英国 AI 安全研究所(AISI)开源的前沿模型评测框架，被 Anthropic、DeepMind、METR 等广泛采用。用“数据集 + solver + scorer”组合，内置 200+ 现成 benchmark。

怎么用

pip 装 inspect-ai，一条命令跑评测；支持工具调用、MCP、多 agent，可把 Claude Code / Codex CLI 等外部 agent 接进来；Docker / K8s 沙箱隔离。

适合场景

想要严谨、可复现、可扩展的自研评测与安全 / 能力测试；红队、沙箱跑不可信代码的场景。

上手命令

```
pip install inspect-ai
export ANTHROPIC_API_KEY=...

inspect eval task.py \
  --model \
    anthropic/claude-sonnet-4-5

# Inspect View 看板
# + VS Code 插件
```

标签 200+ 现成评测 · solver/scorer · 沙箱 · 接外部 agent



HOLISTIC AGENT LEADERBOARD · 含成本榜

HAL

是什么

普林斯顿推出的“默认带成本核算”的第三方 agent 排行榜 + 标准化 harness。可在数百台 VM 上并行跑，自动追踪 token / 美元成本与 trace，且不需改 agent 代码。

怎么用

用 HAL harness 在本地或云端跑评测，自动用 W&B Weave 记录成本与轨迹；结果可上传官方排行榜做横向对比。

适合场景

关心“准确率 vs 成本”的 Pareto 取舍；需要第三方、可复现、成本感知的评测结果。

上手命令

```
# github:
#   princeton-plt/hal-harness

# 1) 本地/云端跑 hal-eval
# 2) Weave 自动记
#   成本 + token + trace
# 3) 提交官方榜对比

# hal.cs.princeton.edu
```

标签 成本感知 · 第三方榜 · 可复现 · Weave 记录



对话 / 客服 AGENT 评测

IntellAgent

是什么

Plurai 开源的多 agent 框架，专评对话型 / 客服 agent。用“策略图建模 + 真实事件生成 + 用户-agent 模拟”自动生成多样化合成基准，做细粒度诊断。

怎么用

给定你的策略(policy)与工具 / API，框架自动生成不同复杂度的对话场景并模拟用户，按策略类别给出弱点诊断。

适合场景

多轮、带政策约束、要调 API 的任务型 / 客服机器人；想定位“在哪些策略上掉链子”。

上手命令

```
# github:
#   plurai-ai/intellagent

pip install -r \
    requirements.txt

# 1) 配置 policy + tools
# 2) 自动生成对话场景
# 3) 模拟用户 + 诊断
```

标签 多 agent 模拟 · 策略图 · 合成基准 · 对话诊断



自动擂台 · LLM 陪审团

AutoArena

是什么

Kolena 开源的自动“擂台”评测工具。核心是 PoLL（多个 LLM 评委组成陪审团），用一组较小的不同家族模型当裁判，比单个大模型更准、更快、更便宜。

怎么用

起本地服务后，上传带 prompt / response 两列的 CSV，配置裁判即可自动两两对打，把胜负换算成 Elo 榜，比较模型 / RAG / prompt 变体。

适合场景

想快速给“模型 / RAG / prompt 变体”排一个可信榜单；做选型与回归对比。

上手命令

```
pip install autoarena
python -m autoarena serve

# 1) 上传 CSV
#      (prompt, response)
# 2) 配置 LLM 裁判 (PoLL)
# 3) 自动对打 -> Elo 榜
```

标签 PoLL 陪审团 · Elo 榜 · CSV 上手 · 选型对比